Dr. Wittawin Susutti

wittawin.sus@kmutt.ac.th

CSS234 WEB PROGRAMMING II

LECTURE 02 API

What is an API?

Definition

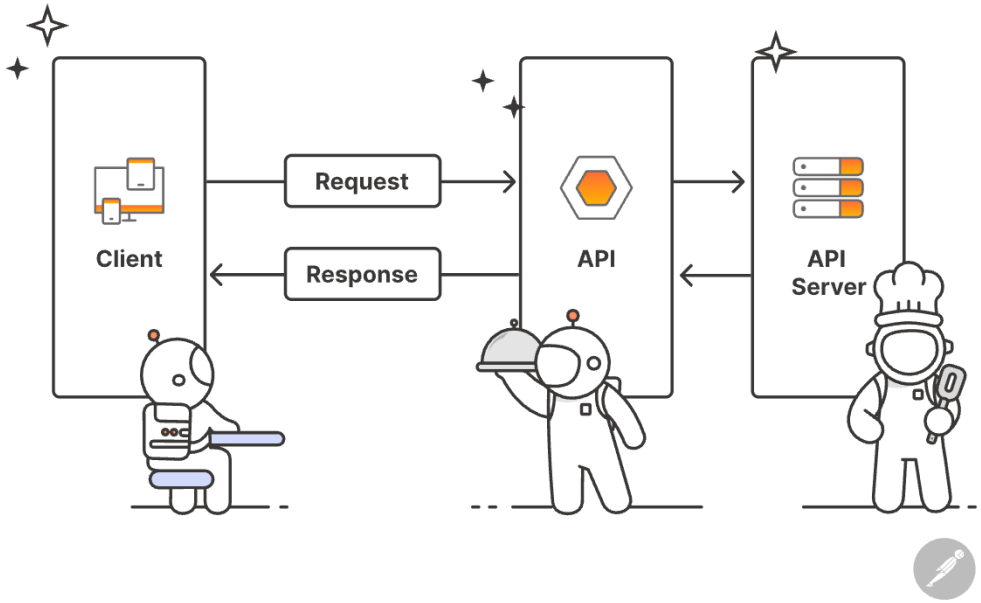
- Application Programming Interface (API), a set of rules that allows different software applications to communicate with each other.

How it Works

- Acts as a middleman that takes a request from a "Client" and delivers a response from a "Server."

Business Value

- Enhances efficiency, enables innovation, and allows for the integration of third-party services (e.g., Google Maps on a delivery app).



Type of APIs

- **Private APIs:** Private APIs (internal APIs) are used to connect different software components within a single organization, and they are not available for third-party use.
- **Public APIs:** Public APIs provide public access to an organization's data, functionality, or services, which third-party developers can integrate into their own applications.
- **Partner APIs:** Partner APIs enable two or more companies to share data or functionality in order to collaborate on a project. They are not available to the general public and therefore leverage authentication mechanisms to ensure they are only used by authorized partners.

Common API Architectural Styles

SOAP (Simple Object Access Protocol)

- A highly structured protocol used for secure, enterprise-level communication.

Restful API

- The most popular API architecture for transferring data over the internet.
- Operations are performed on those resources with standard HTTP methods such as GET, POST, PUT, and DELETE.

gRPC

- A high-performance style often used for communication between microservices.

Webhooks

- Webhooks are used to implement event-driven architectures.
- For instance, when a specific event occurs in an application, such as a payment being made, the application can send an HTTP request to a pre-configured webhook URL with the relevant event data in the request payload.
- The system that receives the webhook can then process the event and take the appropriate action.

GraphQL

- A query language that allows clients to request exactly the data they need.

Understanding SOAP APIs

Strict Protocol

- A formal protocol with rigid rules and standards.

Format

- Exclusively uses **XML** for messaging.

Key Strengths:

- **WS-Security**
  - Built-in high-level security standards.
- **ACID Compliance**
  - Ensures database integrity for complex transactions.
- **Usage**
  - Best suited for financial services, payment gateways, and legacy enterprise systems.

Understanding RESTful APIs

Key Principles

- **Statelessness**
  - Each request must contain all information needed to process it; the server does not store client session data.
- **Uniform Interface**
  - Simplifies the architecture by using standard HTTP methods (GET, POST, PUT, DELETE).
- **Client-Server Decoupling**
  - The client and server evolve independently.
- **Cacheability**
  - Responses can be cached to improve performance.
- **Layered System**
  - The architecture can have multiple layers (e.g., security, load balancing) without affecting the client's interaction.

SOAP vs REST
Side-by-side comparison

Feature	SOAP	REST
Message format	XML only	JSON (most common), XML, YAML
Transport support	HTTP, SMTP, TCP, and more	HTTP only
Built-in features	Security, reliability, transaction support	Implemented manually
Structure	Rigid, contract-driven	Flexible, loosely coupled
Performance	Slower due to larger XML messages	Faster with smaller, lightweight messages
Change management	Difficult—changes require client updates	Easier—supports frequent updates
Common use cases	Banking, healthcare, enterprise backends	Web apps, mobile apps, public APIs

REST vs. SOAP: Key Differences

Flexibility

- REST is highly flexible and lightweight; SOAP is strict and heavy.

Performance:

- REST is generally faster due to smaller payloads (JSON); SOAP has more overhead due to XML parsing.

Security

- SOAP has built-in security (WS-Security), while REST relies on underlying protocols like HTTPS (TLS).

Success Criteria:

- Choose **REST** for web/mobile apps and public APIs.
- Choose **SOAP** for high-security enterprise transactions and banking.

Introduction to REST API

REST Definition

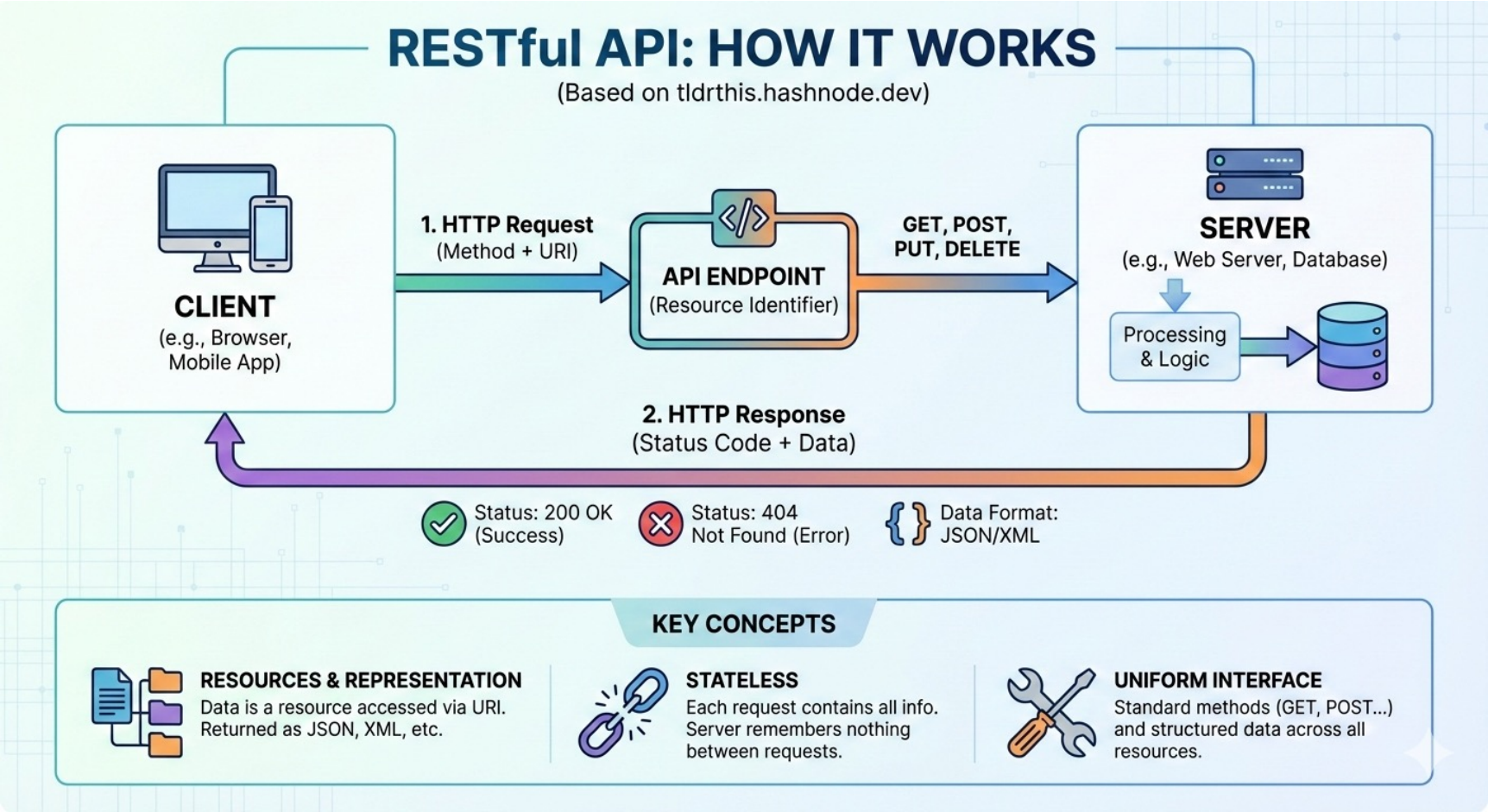
- Stands for Representational State Transfer, an architectural style for providing standards between computer systems on the web.

Core Philosophy

- Focuses on resources, which are identified by URIs, and uses standard HTTP methods to manipulate them.

Key Benefits

- Scalability, flexibility, independence between client and server, and high performance through caching.


RESTful API: HOW IT WORKS
(Based on tldrthis.hashnode.dev)
CLIENT
(e.g., Browser, Mobile App)
1. HTTP Request
(Method + URI)
API ENDPOINT
(Resource Identifier)
GET, POST, PUT, DELETE
SERVER
(e.g., Web Server, Database)
Processing & Logic
2. HTTP Response
(Status Code + Data)
Status: 200 OK (Success)
Status: 404 Not Found (Error)
Data Format: JSON/XML
KEY CONCEPTS
RESOURCES & REPRESENTATION
Data is a resource accessed via URI. Returned as JSON, XML, etc.
STATELESS
Each request contains all info. Server remembers nothing between requests.
UNIFORM INTERFACE
Standard methods (GET, POST...) and structured data across all resources.

URI Design & Best Practices

Nouns, Not Verbs

- Use nouns to represent resources (e.g., /users) instead of verbs (e.g., /getUsers).

Use Plural Nouns

- Keep resource names consistent by using plurals (e.g., /customers/5 instead of /customer/5).

Logical Hierarchy

- Use forward slashes to show relationships (e.g., /authors/45/books).

Lowercase Only

- Use lowercase letters in URIs to avoid confusion (URIs are case-sensitive).

Avoid Special Characters

- Use hyphens (-) for readability; avoid underscores (_) or encoded characters.

Mapping CRUD to HTTP Methods

- **POST (Create):** Used to create a new resource in a collection.
- **GET (Read):** Used to retrieve a specific resource or a collection.
- **PUT (Update/Replace):** Used to update an existing resource by replacing it entirely.
- **PATCH (Partial Update):** Used to update only specific fields of an existing resource.
- **DELETE (Delete):** Used to remove a specific resource from the server.

HTTP Status Codes

2xx Success (The "Everything is Fine" Codes)

- These codes indicate that the client's request was successfully received, understood, and accepted.
- **200 OK**
  - The standard response for successful HTTP requests.
  - Used for GET (returning data), PUT (updating data), or DELETE (if returning the deleted object).
- **201 Created**
  - Specifically for POST requests.
  - It signals that a new resource has been successfully created.
- **204 No Content**
  - The request was successful, but there is no data to send back.
  - This is common for DELETE requests or PUT updates where the client doesn't need to see the object again.

HTTP Status Codes

4xx Client Errors (The "You Did Something Wrong" Codes)

- **400 Bad Request**
  - The server cannot process the request due to client error.
- **401 Unauthorized**
  - The user is not authenticated (need to log in or provide a valid API key/Token).
- **403 Forbidden**
  - The user is authenticated but **does not have permission** to access the specific resource.
- **404 Not Found**
  - The requested resource could not be found.
  - This is frequently used when a specific ID in the URL does not exist in the database.
- **405 Method Not Allowed**
  - The endpoint exists, but the HTTP method used is not supported.
- **429 Too Many Requests**
  - The user has sent too many requests in a given amount of time.

HTTP Status Codes

5xx Server Errors (The "I'm Broken" Codes)

- **500 Internal Server Error**
  - A generic error message when the server encounters an unexpected condition.
- **502 Bad Gateway**
  - The server, while acting as a gateway or proxy, received an invalid response from an upstream server.
- **503 Service Unavailable**
  - The server is currently unable to handle the request due to maintenance or being overloaded.
- **504 Gateway Timeout**
  - The server was acting as a gateway and timed out waiting for a response from the backend.

Professional Tips

- **Be Specific**
  - Never return 200 OK if the request actually failed.
- **Provide a Body for 4xx/5xx**
  - When returning an error, include a JSON body explaining *why* it failed.
  - **Example:** 400 Bad Request

```
{
  "error": "Invalid Input",
  "message": "The 'email' field must be a valid email address."
}
```

- **Use 204 for Deletes**
  - It saves bandwidth and clearly indicates that the resource is gone and there is nothing more to show.

API Versioning Strategies

- **Why Version?**
  - To prevent breaking changes for existing users when updating the API.
- **URI Versioning**
  - Including the version in the URL path (e.g., api.example.com/v1/users).
- **Header Versioning**
  - Specifying the version in a custom request header (e.g., Accept-Version: v1).
- **Media Type Versioning**
  - Using the Accept header to define the version (e.g., Accept: application/vnd.myapi.v1+json).
- **Best Practice**
  - Always version your API from the start to ensure long-term maintainability.

Robust Design & Advanced Concepts

- **Idempotency**
  - Ensure that performing the same request multiple times has the same effect (critical for PUT and DELETE).
- **Error Handling**
  - Use standard HTTP Status Codes (e.g., 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error).
- **Pagination & Filtering**
  - For large datasets, provide ways to limit data (e.g., /books?page=2&limit=20).
- **Security Standards**
  - Implement authentication and authorization using industry standards like OAuth2 or JWT (JSON Web Tokens).

Pagination: Managing Large Data Sets

- Handling large volumes of data requires breaking it into smaller, manageable chunks.
- **Purpose**
  - Improves performance, reduces server load, and minimizes network latency by avoiding "mega-responses."
- **Offset-based Pagination**
  - Uses limit (number of items) and offset (starting point).
  - Simple to implement but can become slow for very large datasets and may miss items if data changes frequently.
- **Cursor-based (Keyset) Pagination**
  - Uses a unique pointer (cursor) to the last item fetched.
  - Highly efficient for large data and real-time streams as it handles data insertions/deletions better than offsets.
- **Standard Headers**
  - Often uses Link headers to provide URLs for "next", "previous", "first", and "last" pages.

Traffic Control

- Techniques to protect the API from abuse, spikes in traffic, and to ensure fair usage.
- **Rate Limiting**
  - Restricts the number of requests a user can make within a specific timeframe (e.g., 100 requests per minute).
  - Essential for preventing DDoS attacks and brute force.
- **Throttling**
  - Controls the *speed* or *frequency* of request processing.
  - It slows down users who exceed their quota rather than blocking them entirely.
- **Debouncing**
  - Often used on the client-side to ensure a function is only triggered after a certain period of inactivity (e.g., waiting for a user to stop typing before making a search API call).
- **Benefits**
  - Enhances security, improves reliability, and helps manage infrastructure costs.

Idempotency: Ensuring Consistency

- A critical concept for reliability, especially when network issues cause request retries.

- **Definition**
  - An operation is idempotent if performing it multiple times produces the same result as performing it once.

- **HTTP Method Mapping**
  - **Idempotent:** GET, HEAD, PUT, DELETE, OPTIONS.
  - **Non-Idempotent:** POST (typically creates a new resource every time).

- **Implementation**
  - Often involves using an **Idempotency-Key** in the header.
  - If the server receives the same key again, it returns the cached response instead of processing the request a second time.

Error Handling

- Clear communication between the server and the client when things go wrong.

- **Standardized Status Codes**
  - **4xx (Client Errors):** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 429 Too Many Requests.
  - **5xx (Server Errors):** 500 Internal Server Error, 503 Service Unavailable.

- **Consistent Error Body**
  - Always return a structured JSON response containing:
    - An internal error code.
    - A human-readable message.
    - (Optional) Links to documentation for troubleshooting.

- **Security Note**
  - Never leak sensitive system details (like stack traces or DB queries) in public error messages.

API Documentation

- Effective API documentation is not just a manual; it is the **interface** of your product.

Benefits

- **Improved Developer Experience (DX)**
  - Clear docs allow internal and external developers to integrate your services in minutes rather than days.
- **Single Source of Truth**
  - It serves as a contract between Frontend and Backend teams, reducing "integration friction."
- **Faster Debugging & Testing**
  - Well-defined endpoints make it easier to isolate bugs and write automated tests.
- **Automation Ready**
  - Standardized documentation (like OpenAPI) allows you to auto-generate Client SDKs, Mock Servers, and Test Suites.
- **Scalability & Governance**
  - It ensures consistency in naming conventions, security protocols, and data structures across a microservices architecture.

How Swagger Works

- **Define**
  - You write a YAML or JSON file.
- **Visualize**
  - Swagger UI reads that file and renders a webpage where you can "Try it out.”
- **Execute**
  - When you click "Execute," Swagger UI sends a real HTTP request to your backend and displays the result.

Best Practices for API Design

- **Use Nouns, Not Verbs**
- **Pluralize Resources**
- **Consistency**
  - If you use `camelCase` for JSON keys, use it everywhere.
- **Version your API**
  - Always include the version in the URL (e.g., `/v1/products`).
- **Meaningful Error Messages**
  - Don't just send `500`.
  - Send a JSON body explaining *why* it failed (e.g., `{ "error": "Invalid ISBN format" }`).
- **Pro Tip:** In a **Design-First** workflow, you should finish this YAML file and get approval from the Frontend team **before** you start writing the actual backend code.

Practice

- Create CRUD API for Book management system with good design of HTTP status code responses.
- Add search and sorting functions.
- Create Swagger for this app.
- You can design your own schema (database connection is not necessary).